

Lab 5: Designing and Training Neural Networks — Logic Gates, Backpropagation and Vanishing Gradient

Course-	M.Tech.	Type- Core	Core
Course Code-	ABAI5005L	Course Name-	Advanced Machine Learning
Year-	2026-27	Semester-	ODD
Date-	18-08-2026	Batch-	1

Scope of this lab: This lab moves from a simple perceptron to a multi-layer neural network and implements the essential components of neural-network learning from scratch using NumPy — covering logic-gate datasets, forward propagation, backpropagation, and the vanishing-gradient problem. Duration: 2 Hours. Software: Python 3.x, Jupyter Notebook / Google Colab. Do not use TensorFlow

Learning Objectives

After completing this laboratory, students will be able to:

- Understand why a neural network needs hidden layers to learn non-linearly separable problems such as XOR.
- Implement forward propagation through a multi-layer neural network using NumPy.
- Derive and implement backpropagation to train a neural network from scratch.
- Train a neural network on the XOR problem using gradient descent.
- Investigate the vanishing-gradient problem in deep networks using sigmoid activation.
- Compare sigmoid and ReLU activations in terms of gradient behaviour and training performance.

Background

Part A: Logic Gates and Linear Separability

AND and OR are linearly separable problems that a single-layer perceptron can learn directly. XOR is not linearly separable, so a single-layer perceptron cannot learn it — a hidden layer is required to transform the input space so that the classes become separable.

Part B: Forward Propagation and Backpropagation

Forward propagation computes the network's output layer by layer: $Z = XW + b$ followed by an activation function such as sigmoid. Backpropagation computes the gradient of the loss with respect to every weight and bias by applying the chain rule backwards from the output layer to the input layer, after which gradient descent is used to update the parameters.

Part C: Vanishing Gradient

In deep networks trained with sigmoid activation, gradients can shrink rapidly as they are propagated backwards through many layers, since the derivative of the sigmoid function is at most 0.25. This makes the early layers of a deep network learn very slowly — a problem known as the vanishing-gradient problem. ReLU activation can help alleviate this problem because its derivative does not saturate for positive inputs.

Required Libraries

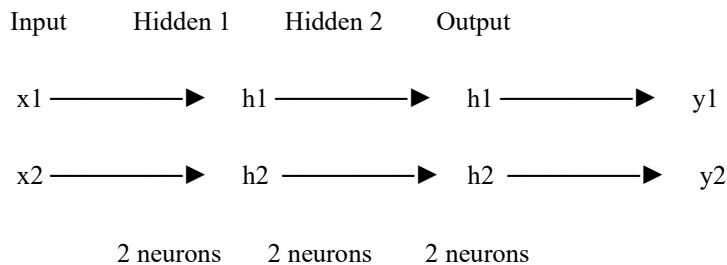
Library	Purpose
numpy	Neural-network calculations (arrays, matrix multiplication, gradients)
matplotlib.pyplot	Visualization (datasets, training loss, gradient magnitude)
sklearn.datasets	Generate the small nonlinear dataset (make_moons)
sklearn.model_selection	Train-test split
sklearn.metrics	Classification accuracy

Question 1: Implement and Analyze Logic Gates

Objective: Understand why a neural network needs hidden layers to learn non-linearly separable problems.

Instructions:

- Create datasets for AND, OR and XOR using the four possible combinations of two binary inputs: [0,0], [0,1], [1,0], [1,1].
- Represent AND, OR and XOR as NumPy arrays.
- Plot the three datasets using matplotlib.
- Determine which of the three problems are linearly separable.
- Explain why a single-layer perceptron can learn AND and OR but fails to learn XOR.
- Design a 2-2-2-2 neural network for XOR, with an input layer, two hidden layers of 2 neurons each, and an output layer of 2 neurons. Include a bias for every neuron in the hidden and output layers.
-



Python / NumPy hints:

- `np.array()`, `np.zeros()`, `np.random.randn()`, `np.random.seed()` to create and initialize data
- `plt.scatter()`, `plt.xlabel()`, `plt.ylabel()`, `plt.title()`, `plt.show()` for plotting
- Note: do not use TensorFlow / Keras / PyTorch for this exercise

Question 2: Implement Forward Propagation

Objective: Understand how input data travels through the 2-2-2-2 network.

Instructions:

- Initialize weights and biases: $W1$ (2×2), $b1$ (1×2), $W2$ (2×2), $b2$ (1×2), $W3$ (2×2), $b3$ (1×2). Use sigmoid activation initially.
- Implement the sigmoid activation function and its derivative.
- Implement forward propagation: $Z1 = XW1 + b1$, $A1 = \sigma(Z1)$, $Z2 = A1W2 + b2$, $A2 = \sigma(Z2)$, $Z3 = A2W3 + b3$, $\hat{Y} = \sigma(Z3)$.
- Print the dimensions of all intermediate variables.

- For one input sample, display the Input, Z1, A1, Z2, A2, Z3, and Output.
- Implement Mean Squared Error, $L = (1/N) \sum (Y - \hat{Y})^2$, and display the initial loss.

Python / NumPy hints:

- *np.exp()* for sigmoid; *np.dot()* or the *@* operator for matrix multiplication
- *np.mean()*, *np.square()* for the loss function
- Pay particular attention to matrix dimensions and bias broadcasting

Question 3: Implement Backpropagation and Train XOR

Objective: Implement the learning mechanism of the neural network rather than using `model.fit()`.

Instructions:

- Starting from the output error, derive and implement the gradients $dW3$, $db3$, $dW2$, $db2$, $dW1$, $db1$ by propagating backwards: Loss $\rightarrow$ Output Layer $\rightarrow$ Hidden Layer 2 $\rightarrow$ Hidden Layer 1.
- Update the parameters using gradient descent: $W \leftarrow W - \eta \cdot dW$ and $b \leftarrow b - \eta \cdot db$, where η is the learning rate.
- Create a training loop for approximately 5,000–10,000 epochs. At each epoch: (1) forward propagation, (2) calculate loss, (3) backpropagation, (4) update weights and biases, (5) store the loss.
- Plot the training loss against epochs.
- After training, predict the four XOR samples and report the network's output, using prediction = 1 if output ≥ 0.5 , otherwise prediction = 0.
- Answer: Why does the network need both hidden layers even though the dataset contains only four samples?

Python / NumPy hints:

- *range()*, *np.mean()*, *np.sum()*, *np.dot()* for the training loop and gradients
- *W.shape*, *b.shape*, *np.linalg.norm()* for debugging
- Important: do not use an optimizer such as Adam or `model.fit()` — implement gradient descent yourself

Results Table — XOR Predictions:

Input	Actual XOR / Predicted / Network Output
0, 0	
0, 1	
1, 0	
1, 1	

Question 4: Small Dataset and Vanishing Gradient

Objective: Connect the small XOR experiment to real machine-learning training and investigate the vanishing-gradient problem.

Part A — Small Nonlinear Dataset

Instructions:

- Generate a small nonlinear dataset using `sklearn.datasets.make_moons()`, with approximately 100 samples. Generate and visualize the dataset.
- Divide the dataset into training and testing sets using `train_test_split()`.

- Train your NumPy neural network on the dataset.
- Calculate the classification accuracy using `accuracy_score()`.

Part B — Investigate Vanishing Gradient

Instructions:

- Make the network deeper, for example $2 \rightarrow 2 \rightarrow 2 \rightarrow 2 \rightarrow 2 \rightarrow 2 \rightarrow 2$, using sigmoid activation in the hidden layers.
- During backpropagation, record the magnitude of the gradient for each layer ($\|dW1\|$, $\|dW2\|$, $\|dW3\|$, ...) using `np.linalg.norm()`.
- Plot gradient magnitude against layer number.
- Replace sigmoid with ReLU in the hidden layers by implementing `relu(z)` and `relu_derivative(z)`, using `np.maximum()`.
- Compare sigmoid and ReLU in terms of early-layer gradient, training behaviour, and final performance in the table below.

Comparison Table:

Activation	Early-layer Gradient	Training Behaviour	Final Performance
Sigmoid			
ReLU			

Sub-questions:

- Why does the gradient become very small in early layers when sigmoid is used?
- Why can ReLU alleviate the vanishing-gradient problem?
- Does ReLU completely eliminate problems associated with gradients? Explain.
- Why is this problem much more important in a deep network than in a shallow network?

Python / NumPy hints:

- `make_moons(n_samples=?, noise=?, random_state=?)` to generate the dataset
- `train_test_split()` to divide the dataset; `accuracy_score()` to evaluate it
- `np.linalg.norm()` to compute gradient magnitude; `np.maximum()` to implement ReLU

Expected Output: Students should include the AND/OR/XOR datasets and plots, an explanation of linear separability, the forward-propagation values for a sample input, the training loss curve, the XOR prediction results, the make_moons dataset and its classification accuracy, the gradient-magnitude plot for the deep sigmoid network, the sigmoid-vs-ReLU comparison, and answers to all sub-questions.

Viva Questions

- Why is a hidden layer necessary to learn the XOR function?
- What is the role of the bias term in a neuron?
- Explain the difference between forward propagation and backpropagation.
- How is the chain rule used in backpropagation?
- What is the effect of the learning rate on gradient descent training?
- What is the vanishing-gradient problem, and why does it occur with sigmoid activation?
- How does ReLU help mitigate the vanishing-gradient problem?

- What are the limitations or drawbacks of using ReLU?
- Why is make_moons used instead of a linearly separable dataset to test the network?
- How does increasing network depth affect training difficulty?

Deliverables

Students must submit:

- Jupyter Notebook (.ipynb) containing all implementations and outputs.
- PDF report including: the AND/OR/XOR dataset plots and linear-separability analysis, the forward-propagation walkthrough, the training loss curve and XOR prediction results, the make_moons classification results, the gradient-magnitude analysis for sigmoid versus ReLU, and answers to all lab and sub-questions.